Formation Vibe Coding condensée

Modalités

- Durée: 1 jour
- Lieu: à distance

Pré-requis

- Cursor installé, configuré avec un compte Cursor
- Bases techniques en programmation et git
- Logiciels: `git`, `npm`, `python`
- Logiciels facultatifs: `docker`

Objectifs pédagogiques

- Installer et configurer Cursor selon votre environnement
- Exploiter efficacement les suggestions IA pour coder plus rapidement
- Utiliser Cursor pour améliorer la qualité du code et automatiser des tâches
- Personnaliser Cursor selon votre workflow et vos préférences
- Comprendre les bonnes pratiques d'interaction de l'IA pour un code robuste et sécurisé

Public visé

Cette formation s'adresse aux développeur·se·s souhaitant optimiser leur productivité grâce à Cursor.

Plan de formation

Horaire	Module	Durée	Contenu
9h00 - 9h30	**Introduction**	30min	Tour de table expert, objectifs, assistance IA vs projet IA
9h30 - 10h00	**Introduction à Cursor**	30min	Installation, configuration, différences VS Copilot, interface
10h00 - 10h15	☕ **PAUSE**	15min	
10h15 - 11h45	**Utilisation experte de l'IA**	90min	5 Compétences du Vibe Coding, Bonnes Pratiques, TP TaskList
11h45 - 12h30	**Anatomie d'un agent partie 1**	45min	LLM+Rules+Skills+Contexte
12h30 - 13h30	🍽️ **PAUSE DÉJEUNER**	60min	
13h30 - 14h15	**Anatomie d'un agent partie 2**	45min	TP AGENTS.md et SKILL.md
14h15 - 15h15	**Workflow & Automatisation**	60min	Tests, Documentation ADR, Gitworkflow, Husky (TP)
15h15 - 16h15	**Sécurité & Risques**	60min	Vulnérabilités, Menaces 2026, Défenses, Audit technique (TP)
16h15 - 16h30	☕ **PAUSE**	15min	
16h30 - 17h30	**Revue de code & Erreurs**	60min	Débogage, Optimisation, Revue PR/MR, Analyse des logs (TP)
17h30 - 18h00	**Conclusion**	30min	Récapitulatif des points clés et Q&A final

Cours

Introduction (30 min)

- Tour de table : expérience, projets IA en cours
- Présentation des objectifs de la formation
- Positionnement niveau expert

⚠ **Avertissement : assistance IA vs projet IA**

Introduction à Cursor et ses fonctionnalités (30 min)

- Présentation de Cursor
- Différences entre Cursor et GitHub Copilot
- Installation et configuration
- Paramétrage des suggestions IA
- Personnalisation de l'interface

Comprendre et utiliser l'IA pour coder efficacement (1H30)

- Comment Cursor génère ses suggestions
- Validation et amélioration des propositions de code
- Exploitation des modes d'interaction avancés avec l'IA
  - **Agent** : mode pour agir directement sur le code ou lancer des scripts
  - **Plan** : mode pour planifier des tâches complexes, mode à favoriser pour les tâches complexes
  - **Ask** : mode pour poser des questions à l'IA par rapport au projet
  - **Debug** : mode pour déboguer du code, l'IA n'ajoutera pas de fonctionnalité, elle corrigera uniquement le code existant
- Utilisation de Cursor pour refactorer intelligemment
- Les 5 Compétences du Vibe Coding
  - **Thinking** : Réflexion et raisonnement assistés par IA
  - **Prompting** : Ingénierie de prompt avancée
  - **Tools** : Maîtrise des outils IA (Cursor, Claude Code, etc.)
  - **Context** : Gestion du contexte et de la mémoire
  - **Review** : Revue critique et validation
- Bonnes Pratiques Entreprise

- **Commiter**: merge rules, commit messages + pre-hook: linting, tests unitaires, tests d'intégration, tests de performance, tests de sécurité…
- **Sécurité** : OWASP Top 10, couverture de code, duplication de code, analyse statique…
- **Performance** : Optimisation assistée par IA
- **Clean Architecture** : Patterns et principes SOLID, architecture hexagonale, DDD…
- **Tests** : Alerte sur le TDD avec assistance IA, alerte sur la volonté des LLM de modifier les tests (ou le code) sans validation
- **Focus points**: compression de contexte, perte d'information, hallucinations, temps de prise en compte des nouveaux fichiers…

TP : application simple de TaskList

Anatomie technique d'un agent (1H30)

```
Agent IA = LLM + Rules + Skills + Contexte + Mode
```

- **LLM** : Moteur de raisonnement (GPT, Claude, Gemini)
- **Rules** : Règles systématiques définies par AGENTS.md
- **Skills** : Boîte à outils du LLM (MCP, API, ligne de commande, prompts etc.) définies par SKILL.md
- **Contexte** : l'ensemble de la fenêtre de conversation actuelle + systèmes de mémoire court terme/long terme
- **Contrôle** : Mode plan/edit/ask/debug

Liste des modèles

Modèle	Type	Commentaire
Gemini 3.1 Pro	High	
Claude-Opus-4.6	High	plus lent mais plus performant
Composer-1.5	High	plus rapide, performance mal connue
Gemini 3.1 Flash	Medium	
Claude-4.6-Sonnet	Medium	
Composer-1	Medium	rapide

ChatGPT-Codex-5.3	Medium	
Kimi K2	Low	Quasiment gratuit

Liste des uses cases

Use case	Mode	Modèle	Commentaire
Lint, debug simple	debug	Composer-1, Gemini-Flash, Haiku, Kimi K2	modèles légers
Commit	agent	Composer-1, Gemini-Flash, Haiku, Kimi K2	
Lancer un script	agent	Composer-1, Gemini-Flash, Haiku, Kimi K2	
Optimiser le code	Plan	Claude-Opus, Gemini-Pro, Composer-1.5	dérouler avec un modèle léger
Executer les tests	agent	Composer-1, Gemini-Flash, Haiku, Kimi K2	
Écrire des tests	agent	Composer-1, Gemini-Flash	
Développer une fonctionnalité	plan	Composer-1, Claude-Sonnet, Gemini-Flash	Modèles moyens
Développer une epic	plan	Claude-Opus, Gemini-Pro, Composer-1.5	dérouler avec un modèle moyen

TP faire rédiger un AGENTS.md et un SKILL.md pour un agent IA

Automatiser son workflow avec Cursor (1H)

- Génération automatique de tests unitaires et de documentation
- Intégration avec Git pour une gestion efficace des commits
- Sécurisation du code
    - éviter les erreurs courantes et détecter les failles avec Husky

TP : via des SKILL.md, mise en place gitworkflow, mise en place des tests, rédaction de la documentation technique (notamment ADR)

Gestion des risques de sécurité avec Cursor (1H)

- Quelles sont les vulnérabilités potentielles générées par Cursor ?
- Détection de code non sécurisé
  - injections SQL
  - failles XSS
  - problèmes d'authentification
- Bonnes pratiques pour valider et sécuriser le code généré par Cursor
- Intégration avec des outils de vérification de sécurit
  - SonarQube
  - Snyk
- Menaces 2026
  - **Prompt Injection** : Manipulation via prompts malveillants
  - **Hallucination Attacks** : Exploitation des hallucinations (risque de plus en plus faible, les modèles progressent)
- Exploitations 2026
  - **Data Leakage** : Fuite de données sensibles
  - **Agent Hijacking** : Prise de contrôle de l'agent
- Défenses pratiques
  - **Input Validation** : Validation des entrées, bloquer les recherches web, contrôler le code récupéré
  - **Output Filtering** : Validation des sorties, lire le code généré et les commentaires de l'agent
  - **Sandboxing** : Isolation des agents, peu pratique mais possible si le contexte le nécessite
  - **Monitoring** : Surveillance continue via surveillance de l'activité réseau de l'agent (Firewall, Proxy, etc.)

TP : auditer un logiciel existant et proposer des améliorations en termes de sécurité

Revue de code avec Cursor (1H)

- Débogage assisté par IA
  - Analyse automatique des stack traces
  - Détection de patterns d'erreurs
  - Suggestions de fixes contextualisés
- Optimisation de code
  - Performance (complexité algorithmique, optimisations)

    - Lisibilité (refactoring, naming)
    - Maintenabilité (découplage, SOLID)
- Génération de tests
    - Tests unitaires
    - Tests d'intégration
    - Tests end-to-end
    - Contrôle de la cohérence des données
    - Génération de données de test
- Revue de code assistée
    - Relectures de Pull Request / Merge Request
    - Standards et conventions
    - Sécurité (OWASP)
    - Performance
    - Maintenabilité
    - Documentation

TP : mise en place d'un workflow de revue de code avec Cursor et Husky

Gestion et correction des erreurs générées par Cursor (1H)

- Identifier les erreurs courantes générées par Cursor
- Analyse des erreurs de logique, syntaxe et compatibilité
- Exploitation des logs et retours de Cursor

TP: analyse des logs de cursor

Conclusion (30 min)

- Récapitulatif des points clés
- Questions et réponses

Date de mise à jour: 05/03/2026
Auteur: Philippe PARY (philippe.pary@astek.net)